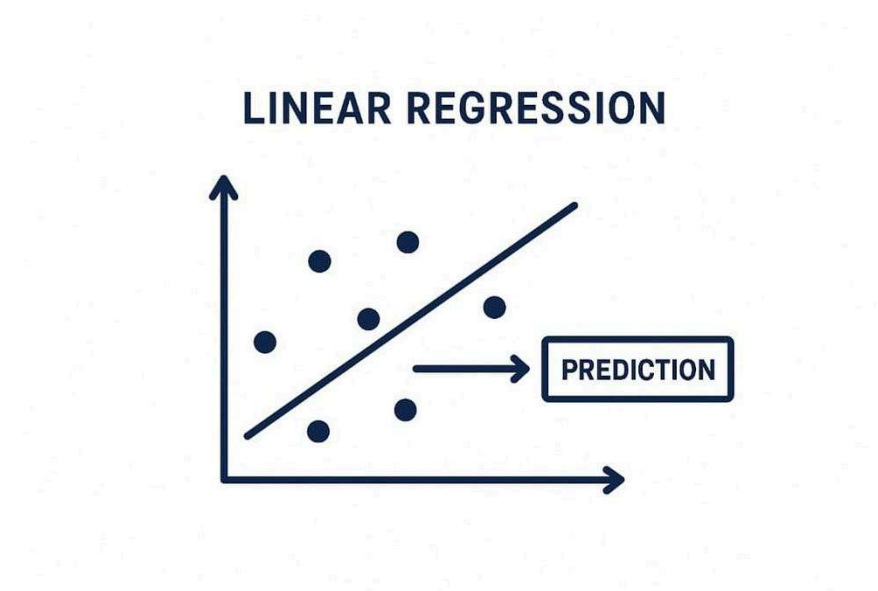


First step into Machine Learning: Linear Regression

Understanding how a simple line can make powerful predictions

If you've ever taken your first steps into Machine Learning, you've probably met Linear Regression. It's the "Hello World" of Machine Learning - simple, yet incredibly powerful.



Why should you learn Linear Regression first?

1. It is easy to understand and interpret and that's why it is starting point for learning machine learning.
2. Many algorithms in Machine Learning like Logistic Regression and Neural Networks, build on the concept of Linear Regression.

In this blog ...

First, we will try to understand how Linear Regression works and the math behind it and after it we will make our own Linear Regression model using scikit-learn Library. After that, we'll look at its limitations and how we can address them.

Imagine that you're trying to predict your exam marks based on the number of hours you study. So, you collect some data from your classmates that how much they studied in previous exams and what scores they got. Then you plot those data points on a graph, and notice a pattern that as study hour increase, exam marks tend to increase too.

The question is:

Can we find a line that best fits this data and helps us predict marks for any number of study hours?

That's exactly what Linear Regression does.

What is Linear Regression?

Linear Regression finds the best-fitting straight line through a set of data points. This line represents that how one data item affects another. For example, how study hours affect marks.

The relationship is written as:

$$y = m \cdot x + b$$

where:

- y: The value that we want to predict (Here Marks)
- x: Input value (Here Study hours)
- m: Slope of the line (how much marks increase per study hour)
- b: intercept (Value of y when x is 0)

Let's say that we have this small dataset:

Study Hours	Marks
0.5	28
1	36
1.5	39
2	47
2.5	52
3	54
3.5	62
4	66
4.5	73
5	74
5.5	80
6	84

If we plot these points, they form an upward trend. We can almost see a line running through them. Now, Linear Regression helps us find the exact line that best fits.

We'll find that line using the formula:

$$y = m \cdot x + b$$

Step 1: Calculate Slope(m) and Intercept(b)

To put slope and Intercept value we have to calculate it first.

Equation for Slope:

$$m = \frac{n \sum xy - \sum x \sum y}{n \sum x^2 - (\sum x)^2}$$

Equation for Intercept:

$$b = \frac{\sum y \sum x^2 - \sum x \sum xy}{n \sum x^2 - (\sum x)^2}$$

Through that we will get our slope and intercept that we can put on our equation of Linear Regression.

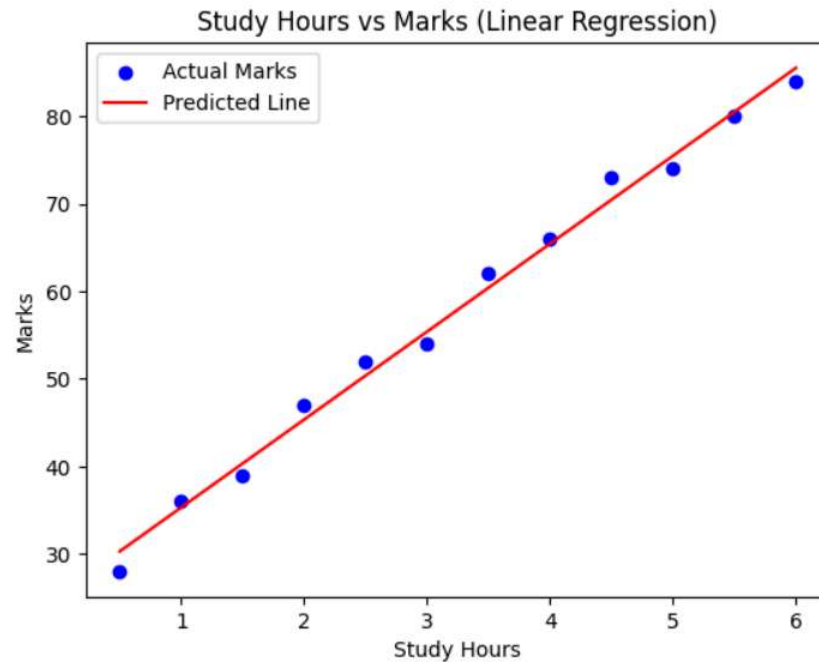
Step 2: Make Predictions

Study Hours	Actual Marks	Predicted Marks
0.5	28	30
1	36	35
1.5	39	40
2	47	45
2.5	52	50
3	54	55
3.5	62	60
4	66	65
4.5	73	70
5	74	76
5.5	80	81
6	84	86

As per our model if a person studied for seven hours, then he will score 96 marks.

Here Predicted values are little differ from actual value. There can be many reasons like,

1. Linear Regression assumes a linear relationship between the input and output. But in reality, data often isn't perfectly linear. So, the best the model can do is draw a line that minimizes the overall error.
2. Our given data is not so huge, but real-world data always contains noise, it can have some random variations, measurement errors, or unknown factors that might affect the output. These cause actual y values to deviate from the predicted ones even if the model is good.



The Regression line represents our model's predictions, simple yet powerful.

By given equation of slope and intercept, our value of slope is 10.05 and value of intercept is 25.26.

So, if we put this into linear equation and then try to find marks value for seven hours of study then we will get,

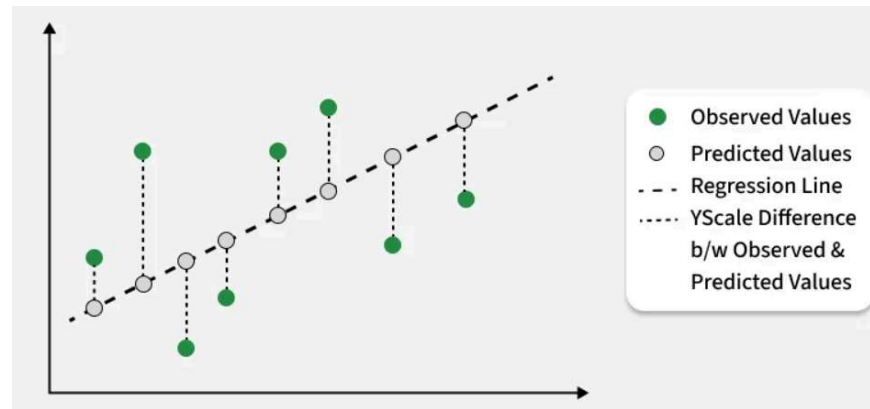
$$y = 10.05(7) + 25.26 = 95.61$$

And if we round off this value the predicted marks will be 96.

Understanding the Best Fit

The question might arise that why we choose this particular line?

Reason is that it's the one that minimum error. Error is the difference between what our model predicts and what the actual data says.



We measure this with the Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

where Y_i is the actual value and $\hat{Y}_i$ is predicted value for each observation.

A smaller MSE means our line fits better.

Code of Linear Regression for above tiny dataset:

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.linear_model import LinearRegression

data = {
    'Study Hours': [0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0],
    'Marks': [28, 36, 39, 47, 52, 54, 62, 66, 73, 78]
}
x = pd.Series(data['Study Hours'])
y = pd.Series(data['Marks'])
X = x.values.reshape(-1, 1)

model = LinearRegression()
model.fit(X,y)

y_hat = model.predict(X)
y_hat_int = np.round(y_hat).astype(int)
print(y_hat_int)

print(f"Intercept: {model.intercept_:.2f}")
print(f"Slope: {model.coef_[0]:.2f}")
```

```
# Plot
plt.scatter(x, y, color='blue', label='Actual Marks')
plt.plot(x, y_hat, color='red', label='Predicted Marks')
plt.title("Study Hours vs Marks (Linear Regression)")
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.legend()
plt.show()

model.predict([[7]])
```

How Do We Measure How Good the Line Is?

Once we build a model, we need a way to evaluate how well it performs. One common metric is **R-squared** (R^2). It tells us how much of the variation in the output (marks) is explained by the input (study hours).

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

where, $\bar{y}$ is mean of actual values.

Here,

- R^2 close to 1 means the model explains most of the variation.
- R^2 close to 0 means the model doesn't explain the variation well.

In simple terms, higher R^2 means our line does a good job.

When Linear Regression Might Not Work

Linear Regression assumes:

1. The relationship between variables is roughly a straight line.
2. There are no extreme outliers affecting the trend.
3. The spread of errors stays roughly consistent.

Limitations of Linear Regression

Linear Regression is great for learning, but it has some clear boundaries.

1. **It assumes the relationship is a straight line**

If the real pattern has curves or behaves in a more complex way, the model won't capture it well because it can only learn straight-line relationships.

2. **Outliers can mess it up**

One or two unusual data points can pull the line in the wrong direction and affect all predictions.

3. **It expects the errors to stay steady**

The difference between actual values and predicted values should stay roughly the same across all inputs. If errors start growing with larger values, the model becomes unreliable.

4. **It doesn't handle complex interactions well**

If multiple factors influence the result in a non-simple way, Linear Regression starts to struggle. Real life is often more complicated than a single straight line.

5. **It may not generalize well**

If the training data doesn't represent the real world properly, the predictions will be off when used on new data.

How to handle this limitation

1. If the relationship is not linear, we can apply transformations to make it closer to a straight line. If the data shows a clear curve, Polynomial Regression is a better choice.
2. If outliers are pulling the model away from the true pattern, we can either remove the incorrect or extreme data points or use robust regression methods that are less sensitive to outliers.
3. If the errors increase or decrease in a pattern, we can try transforming the data (like using a log transform) or use

Weighted Linear Regression, which gives less importance to points with larger errors.

4. When multiple inputs affect the outcome, we move to Multiple Linear Regression and choose the features thoughtfully, so the model doesn't become noisy.
5. If the model performs poorly on new data, we should use a proper train-test split, apply cross-validation, and make sure the data quality is good.

Regularization

When we create a model, sometimes it learns too much from the training data. It starts remembering noise, mistakes, and random patterns.

This is called Overfitting. When a model overfits, it learns the training data too well, so it works great there but performs poorly on new or unseen data. Regularization helps to prevent that. Think of it like **telling your model to stay simple and not overthink**.

L1 Regularization (Lasso Regression)

- L1 regularization, also known as Lasso (Least Absolute Shrinkage and Selection Operator), adds a penalty equal to the absolute value of the coefficients' magnitudes to the loss function of a linear regression model.

- This technique encourages sparsity, meaning it drives some coefficients to exactly zero, effectively removing unimportant features from the model.
- L1 is useful when you expect that only a few features are significant, and it helps interpretability by performing automatic feature selection.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m |w_i|$$

Equation of Lasso Regression

Where:

- λ : Regularization parameter that controls the strength of regularization
- w_i : Coefficients of the features

Code for Lasso Regression:

```
from sklearn.linear_model import Lasso
from sklearn.model_selection import train_test_s
from sklearn.datasets import make_regression
from sklearn.metrics import mean_absolute_error

# Generate synthetic data with 150 samples and 7
```

```
X, y = make_regression(n_samples=150, n_features
X_train, X_test, y_train, y_test = train_test_sp

# Initialize Lasso regression model with a diffe
lasso = Lasso(alpha=0.05) # Here alpha represent

lasso.fit(X_train, y_train)

y_pred = lasso.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
print(f"Mean Absolute Error: {mae}")

# Display the learned coefficients
print("Model Coefficients:", lasso.coef_)
```

L2 Regularization (Ridge)

- L2 regularization, known as Ridge regression, penalizes the sum of the squared values of the coefficients.
- Unlike L1, L2 shrinks all coefficients smoothly toward zero but rarely eliminates any completely (they almost never reach exactly zero).
- L2 regularization is especially helpful when dealing with multicollinearity among features, reducing sensitivity to individual features and promoting model stability.

$$\text{Cost} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{i=1}^m w_i^2$$

Equation of Ridge Regression

Code for Ridge Regression:

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression
from sklearn.metrics import mean_absolute_error

# Generate synthetic data with 150 samples and 7 features
X, y = make_regression(n_samples=150, n_features=7, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize Ridge regression model with a chosen alpha value
ridge = Ridge(alpha=0.05)

ridge.fit(X_train, y_train)

y_pred = ridge.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
print(f"Mean Absolute Error: {mae}")
```



```
# Display the learned coefficients  
print("Model Coefficients:", ridge.coef_)
```

L1 Regularization:
Cutting entire weeds
(features)



L2 Regularization:
Trimming all plants
(features) but leaving
them alive



Difference between L1 and L2 Regularization

Linear Regression is not just a beginner's model. It is the foundation of how ML models learn patterns and make predictions. Once you understand this, advanced topics like Gradient Descent and Neural Networks become easier.

What you can try next:

- Try using a different dataset. It could be your own study record, fitness routine, expenses, anything that forms a pattern.
- Change the input values and see how the predicted results shift. This helps you understand how the model responds.
- Experiment with different *alpha* values in Lasso and Ridge. Notice how the coefficients shrink or even drop to zero. This will give you a feel for how regularization controls model complexity.